Recursive Language Models on Local Hardware: A Transfer Study

DRAFT — H1 concluded; H2–H7 pending; not for citation

Aaron Kingsley Clark*

Applied Artificial Intelligence
Eastern University, St. Davids, PA, USA

Draft of October 2026

Abstract

Recursive Language Models (RLMs) place a long input in a programming environment as a variable and let a language model inspect it with code and call itself on pieces of it, instead of reading the whole input in one prompt. The method was introduced and evaluated with frontier models. We ask whether it carries over to a locally served, quantized open-weight model on shared, serial hardware. This is a transfer study: one model, one serving stack, eight synthetic tasks, and the claims stop there. We contribute (i) ReCLamO-Harness, an open-source, clean-room RLM harness built for local serving, with per-role endpoints, hard sub-call caps, context compaction that guarantees every request fits the window, a forced finish on every limit, and a compatibility mode that runs the original authors' fine-tuned RLM-Qwen3-8B checkpoint; (ii) an evaluation set of eight long-context task generators written, blind to the harness, by eight non-Anthropic models, together with three rounds of audits that found and repaired answer-key and grader defects, and a scorer battery that runs in continuous integration; and (iii) a pre-registered evaluation plan comparing the harness, under two prompt and limit configurations, with a single-call baseline at 32K-token and 128K-token windows, with one primary paired comparison per input size, a power-based target of 320 paired questions, and pre-registered interim looks. A pilot on 16 questions per cell taught us that two measurement errors, selecting the baseline's better attempt with the answer key and defective graders, moved results by as much as the effects under study; we report those lessons separately and draw no conclusions from the pilot. The primary comparison (H1) is concluded at both sizes on 320 confirmatory pairs each, after a scorer defect found mid-study was disclosed and corrected: at the small size the single-call baseline is more accurate (203 vs 229 exact, $z = -3.11$, $p = .0025$), at the medium size the harness is (177 vs 162, $z = +2.19$, $p = .040$), and each result rests on one or two tasks, so neither supports a general claim in either direction. The harness finished 71% of small and 63% of medium questions against the baseline's 100% and 74%, at roughly twice the wall-clock time at small size. The secondary hypotheses (configuration, delegation,

*ORCID 0000-0002-6831-2663. Correspondence: aaron.clark3@eastern.edu.

beyond-window completion, sampling, and two non-recursive baselines) are pre-registered and pending.

Keywords: recursive language models; long-context reasoning; evaluation methodology; local inference; reproducibility

1 Introduction

Language models degrade as their inputs grow, and they cannot read inputs longer than their context window at all. Recursive Language Models [25] address both problems with one move: the input is stored as a variable in a persistent Python REPL, and the model writes code to peek at it, split it, search it and pass pieces of it to fresh language-model calls (`llm_query`). The model sees only the short outputs of its own code. The original paper reports that an RLM built on GPT-5 handles inputs far beyond that model's window and outperforms the base model and common scaffolds on several long-context benchmarks at comparable cost, and that a small fine-tuned model, RLM-Qwen3-8B, improves on its base model.

Those results use frontier models served by commercial APIs, with sub-calls answered by capable models. Many practitioners instead run a quantized open-weight model on hardware they own, one or two requests at a time. This paper asks a narrow, practical question: *on such a setup, does an RLM harness answer long-context questions more accurately than simply giving the model the whole document, and at what cost?* It is a transfer study, not a new method and not a replication: the claims are bounded to this model, this serving stack and these eight tasks, and only the LongBench Pro reproduction planned in Section 10 would be called a replication. The study is also a record of how easily such a comparison is mismeasured: in a pilot, our own measurement errors moved the result as much as the method did.

We make four contributions, and defer a fifth until the data are in.

1. **An open-source harness for local serving** (Section 3). ReCLamO-Harness [8] is a clean-room implementation of the RLM loop after `rlm-minimal` and `rlm` [22, 23], tuned for Qwen-family models on a serial server: per-role endpoints (a planner and a reader on different servers), hard sub-call caps enforced in code, compaction that guarantees each request fits the window, a forced finish on every limit, and an `upstream-rlm-v0` mode that speaks the original scaffold's format so the released RLM-Qwen3-8B checkpoint [12] can plan without retraining.

2. **An independently authored, audited evaluation set** (Section 4). Eight task generators were written by eight non-Anthropic models, each given only a short brief and no access to the harness. Running them exposed task and grader defects that the generators' own self-tests could not catch; three audit rounds repaired them, and a scorer battery of 2,111 probes now runs in CI.

3. **A pre-registered evaluation design** (Section 5): paired arms on identical questions, one primary comparison per size sized by a power calculation from the pilot's discordance rate, interim looks with a corrected threshold, completion reported separately from accuracy, and a phased plan of validity checks, ablations, oracle interventions and coverage tests.

4. **Measurement lessons from a pilot** (Section 7), reported as such: a baseline that was best-of-2 selected on the answer key, grader bugs that cost one arm more than another, a prompt patch copied from the paper that suppressed the very behaviour it was meant to temper, and conditions under which the paper's 8B planner could not be fairly judged.

5. **The primary result** (Section 6): H1 on 320 confirmatory pairs per size, reported twice, as the pre-registered procedure recorded it under the original scorers and as recomputed under corrected scorers after a one-directional scorer defect was found, with the per-task table, leave-one-task-out estimates, completion, cost and latency beside every number. The two sizes point in opposite directions and each rests on one or two tasks. H2–H7 are pending.

2 Background and related work

Recursive Language Models. Zhang et al. [25] define an RLM as an inference-time scaffold in which the prompt is part of the environment rather than the network's input. In their instantiation the root model writes REPL code over a `context` variable, calls sub-models with `llm_query`, and ends by marking a variable or string as final (`FINAL`/`FINAL_VAR`). They evaluate on S-NIAH, built after the single-needle task of RULER [11]; OOLONG and the derived OOLONG-Pairs [2]; BrowseComp-Plus [4]; and code question answering from LongBench v2 [1]. They fine-tune Qwen3-8B [20] on trajectories from RLM(Qwen3-Coder-480B) on LongBench Pro [5] to obtain RLM-Qwen3-8B. Their appendix of negative results is directly relevant here: small models without strong coding ability struggled as RLMs, thinking models ran out of output tokens, sequential sub-calls were slow, and Qwen3-Coder needed an added warning not to make "thousands of LM subcalls for basic tasks". A later post by the same group argues that harnesses can make the individual model calls inside them in-distribution, a form of compositional generalization [24]. The authors also maintain the reference implementations [22, 23] that ours follows.

Long-context evaluation. Goldman et al. [9] argue that many long-context benchmarks reduce to retrieval, and that difficulty should be characterised by how dispersed and how voluminous the needed information is; the RLM paper uses the same lens to explain why GPT-5 degrades faster on OOLONG and OOLONG-Pairs than on S-NIAH. Context degradation within the window ("context rot") is documented by Hong et al. [10]. Latent-structure tasks beyond haystacks are studied by Vodrahalli et al. [17]. [**TODO:** Cite "lost in the middle" (Liu et al.) and LongBench v1 only after verifying the references; neither is in the source reference lists used for this draft.]

Baselines the paper compares with. The RLM paper compares against a retrieval agent using BM25 [14], a CodeAct agent [18], and summary or compaction agents in the line of ReSum [19], context folding [16] and MemAgent [21]. Explicit memory hierarchies [13] and interactive reading [3] are related ways to manage context. Recursive delegation without a symbolic handle to the input appears in THREAD [15] and ReDel [26]. This draft does not yet include a retrieval or compaction baseline; both are planned (Section 10).

3 The ReCLamO harness

ReCLamO-Harness is about 5,300 lines of Python under Apache-2.0. It was written from the paper's description and the MIT-licensed reference code; where text is copied verbatim (the upstream prompt for the compatibility mode, and the upstream orchestrator addendum), the MIT notice sits beside it.

3.1 The loop

The context (a string, a list, or a dictionary of files) is loaded into a persistent REPL as `context`. The system prompt describes the context's type, size and chunk lengths, the REPL tools and the ways to finish. Each root turn is one model call. Fenced `repl` blocks in the reply are executed in order; their outputs, truncated to a configured length, are appended to the history. Inside the REPL the model can call `llm_query(prompt)`, `llm_query_batched(prompts)`, and, when depth > 1 is allowed, `rlm_query(question, data)`, which starts a nested RLM on `data`. The run ends when the model writes `FINAL(...)` or `FINAL_VAR(name)`, or sets `answer["ready"] = True`. A final that arrives in the same reply as unexecuted code, or reads like a plan, is rejected once with a note. Thinking (reasoning) text is logged but never fed back into the history. An optional tool-calling protocol replaces fences and `FINAL` text with `execute_python` and `final_answer` functions.

3.2 Limits, compaction and forced finish

Three classes of limit bound a run, and every one of them ends in an answer rather than an exception.

- **Sub-call caps** per code block and per run are enforced in code. A call past a cap raises an error inside the REPL, which the model sees.
- **Context budget.** Every root request must fit under context_tokens $-$ max_tokens$_{\text{root}}$ $-$ 0.1 context_tokens estimated tokens (never less than a quarter of the window), estimating one token per digit and 3.5 characters per token otherwise. Before each turn, REPL outputs older than the last four are stubbed and, if needed, summarised with one sub-call; if the request still does not fit, the remaining outputs are shrunk largest-first around a stub that points to the full text in a REPL `history` variable; if it still cannot fit, the request is not sent and the run takes the forced finish.
- **Turns, time and errors.** When the turn cap, the wall-clock limit (`max_timeout`) or the consecutive-error limit is reached, a single *forced finish* call shows the model the current answer dictionary, the values of common answer variables and the end of the last REPL output, and asks for `FINAL` or `FINAL_VAR`. Fallbacks, in order, are the answer dictionary, an answer variable, and the reply's prose with code and thinking removed; code is never returned as an answer. Every request is capped by the time left, so a run ends within `max_timeout` plus 60 seconds plus about one second.

Several of these behaviours are fixes for defects found by live runs: a forced finish that returned a stale REPL variable or a code block as the answer; an error limit that discarded a run's work; a

Table 1: The two harness configurations. Sources are the RLM paper's appendices [25] and the upstream code [23]. Sub-call size on `pluto` assumes a 32,768-token reader window.

Setting	v0.1	v0.2
Stance on sub-calls	"sub-calls are expensive ... never 1000"	"strongly encouraged"; caps stated as facts
Planning guidance	none	orchestrator addendum (when to delegate and when not)
Advertised sub-call size	12,000 characters	$(\text{window} - \text{max_tokens}_{\text{sub}}) \times 0.85 \times 3$ characters
Root turns	20	30
REPL output shown	2,000 characters	20,000 characters
Root reply cap	4,096 tokens	8,192 tokens with thinking
Sub-call reply cap	2,048 tokens	4,096 tokens
Sub-calls per run / per block	64 / 24	256 / 64
Request timeout / run limit	300 s / 1,800 s	900 s / 3,600 s

time limit that could be overrun by a turn; and compaction that did not reserve room for the reply. Each is documented in the repository's issue tracker [8].

3.3 Local serving: per-role endpoints and concurrency

The root (planner) and sub (reader) roles may use different servers, keys and concurrency limits; each distinct endpoint gets its own client and semaphore, so sub-calls to a slow reader are not queued behind planner turns on another host. `llm_query_batched` runs up to the reader's concurrency at once. On our main server that is two parallel slots.

3.4 Compatibility with the paper's RLM-Qwen3-8B

The released RLM-Qwen3-8B model card states that the model "was trained on trajectories produced using a fixed system prompt" and "assumes the environment/scaffold from our RLM repo" [12]. The `upstream-rlm-v0` planner style reproduces that scaffold: the paper's Qwen3-8B/32K system prompt [25, App. C.1], the context metadata sent as an assistant message, the per-turn user prompt, `repl` fences, REPL output formatting cut at 20,000 characters, and line-start `FINAL` parsing, as in upstream `rlm` v1.0.0 [23]. Our sandbox, sub-call caps, compaction and forced finish stay on. The exact training prompt is not public, so parts of this reconstruction are approximate; the repository documents each choice.

3.5 Two prompt and limit configurations

Table 1 lists the two configurations compared in this study. Version 0.1 copied the paper's Qwen-specific warning against over-calling and imposed conservative limits. Version 0.2 follows the paper's main (GPT-5) prompt on delegation, adds the upstream "orchestrator, not solver" addendum, derives the advertised sub-call size from the reader's window, and lifts limits we had imposed ourselves. Both are selectable, so v0.1 results remain reproducible.

3.6 Security

Generated code runs in a separate `python -I` process with a scrubbed environment (no API keys), private protocol descriptors, and a per-execution timeout; an optional Docker sandbox adds `-network none`, a read-only root filesystem and a non-root user. The API key never reaches the REPL or the logs.

3.7 Logging

Every run writes a JSONL trajectory: metadata (never the key), each turn's content, reasoning, code, outputs, notes and the serving endpoint and model, each sub-call's sizes and latency, compaction and overflow events, and how the run ended. An optional file records each root turn in the supervised-fine-tuning format the paper used for RLM-Qwen3-8B.

4 An independently authored evaluation set

Why. The harness, its prompts and its first tests were all written with the help of Claude models (Section 10). On those self-authored tests (a million-line needle search, a synthetic ticket-classification task in the style of OOLONG, and two-hop questions over a long digest) the harness looked nearly perfect. Tests written by the harness's authors are likely to be shaped to it, so we commissioned tests from models that had no part in building it.

How. On 2026-10-07 a one-page brief went to eight non-Anthropic models in the author's multi-model review harness [7], one lane at a time. Each lane saw only the brief: no repository, no harness prompt and no other lane's answer. The brief asked for one deterministic, standard-library Python generator with `generate(seed, size)` and `score(answer, truth)`, producing roughly 60,000, 300,000 and 1,200,000 characters of natural-looking text whose answer requires breadth across the document and resists keyword search. A lane running a Qwen model was excluded because the system under test is also a Qwen-branded model. Table 2 lists the eight tasks. The generators are kept byte-for-byte as delivered; every repair lives in a separate file with its provenance, and every version is pinned by SHA-256 in a manifest that CI checks.

Validation and repair. Each generator was first checked in a network-less container: its own self-test, the requested sizes, determinism within and across processes, seed sensitivity, and that the true answer scores 1 and the empty answer scores below 1. Five originals failed (a Python-version syntax error, a degenerate small size, dependence on hash randomisation, a scorer that split numbers, and contexts far below the requested size). Each failure went back to its author model; in parallel, MiniMax-M3, another non-Anthropic model, repaired all five. The author's fix was used when it passed, preserving independent authorship. Later rounds found what these checks could not:

- **Round 2** (found by the first evaluation run): one key described a different item from the one asked about, because question and answer drew separate random choices; one task never stated

Table 2: The eight tasks. Each was written by a different non-Anthropic model, named here by its lane in the review harness.

Lane	Author model	Question
GLaDOS	xAI Grok 4.6	Net authorized amount and certifying member for the legal successor of a renamed project, under bylaws and an SOP
SHODAN	OpenAI GPT-6 Astra	Earned freight credit per office after signed corrections, custody findings and agreement terms
TheDixieFlatline	Google Gemini 3.1 Pro	Final room of an asset through renames, transfers and corrections
Cerebex	Z.ai GLM-5.3 Flash	Travel total after amendments and reversals
Neuromancer	DeepSeek V4 Flash	March travel reimbursed after adjustments and denials
SELMA	NVIDIA Nemotron 3 Super	Final owner of a review after reassignments in an email thread
MasterControl	Mistral Medium 3.1	The one employee flagged for two audit issues, and their total
Multivac	Nous Hermes 4 405B	Status and description of a requirement after merges and splits

the starting state, so some answers were underivable; one task's "unassigned" event produced no text; and one scorer rejected correct answers written as sentences.

- **Round 3** (an independent audit that solved every task from the text before opening any key, plus 1,701 scorer probes): one key filtered on attributes never stated in the text; one task's visible dates contradicted its event order; one correction email was never emitted; and in all eight tasks, correct answers in ordinary formats were marked down while hedged answers naming two candidates got full credit (251 flags).
- **Round 3.1** (found by the pilot): one scorer gave 0.2 to a correct answer in an `ID: description -- Status: X` shape. The fix touched only `score()`; contexts, questions and keys stayed byte-identical. The scorer battery grew to 2,111 probes with zero flags and runs in CI.

Residual properties remain and are reported, not hidden: one task's key is "Unassigned" on about 62% of seeds, and another's key is zero on about 26%.

5 Evaluation design

5.1 System under test

The main model is distributed as Qwen3.8-Flash-Next, in an "abliterated" (refusal-removed) variant by Huihui.ai and a UD-Q4_K_XL quantization by Unsloth. Despite the name, its GGUF metadata declares the architecture `qwen4exp`, "A Preview of the Qwen4 Architecture": a mixture-of-experts model with 512 experts (10 active), 48 layers and a 262,144-token native context. It is therefore not a Qwen3 model, and any comparison with the RLM paper's Qwen3 results is cross-architecture. It is served by the Strata inference server on `pluto`, a workstation with an NVIDIA V100 (32 GB), a P100 (16 GB) and an RTX 3060 (12 GB, on an external PCIe dock). Since 2026-10-08 Strata runs

Table 3: Arms. The plain arms differ only in window; the harness arms differ from each other in configuration (Table 1) and, for v0.2, window.

Arm	Definition
Plain 32K	One call, 32,768-token window (usable 24,576 after the reply reserve)
Plain 128K	One call, 131,072-token window, 1,800 s request timeout
Harness v0.1	v0.1 prompt and limits, 32,768-token root window
Harness v0.2 32K	v0.2 prompt and limits, 32,768-token root window (pilot only)
Harness v0.2 128K	v0.2 prompt and limits, 131,072-token root window

with an int8 KV cache, a 65,536-token resident cache and two parallel slots; it accepts prompts beyond the resident cache at reduced prefill speed. The weight files (sha256 and the Hugging Face revision), the draft head, the tokenizer and chat-template hashes, the engine commit with its one local patch, the build flags, the full server command line and the hardware are recorded in `paper/provenance.md`. The same model serves as planner and reader. Root turns use thinking; sub-calls do not. Sampling follows the Qwen3 model-card presets (root: temperature 0.6, top-p 0.95, top-k 20, min-p 0; sub-calls: 0.7, 0.8, 20, presence penalty 1.0), not the defaults embedded in the GGUF (temperature 1.0, top-p 0.95, top-k 20). The full grid runs on the presets first. The same grid is then repeated with the embedded defaults, and its results are kept separate (H5).

5.2 Arms

All arms answer the same questions (Table 3). "32K" is 32,768 tokens and "128K" is 131,072 tokens. The plain baseline is a single call with the whole document in the prompt, thinking as the harness's root role sets it, and a 16,384-token output cap; a document that does not fit the usable window is recorded as *does not fit* and scores 0. There is no selection between attempts.

5.3 Questions and sample size

Each cell is one task, size and seed. Small, medium and large contexts are about 60K, 300K and 1.2M characters; in the pilot, small documents were 16K–27K estimated tokens and medium ones 86K–148K; large documents run about 340K–610K (seed 0). Seeds 0 and 1 were used during development and are excluded. Seeds 2–3 (16 per cell) form the pilot reported in Section 7.

Power. McNemar's test uses only discordant pairs. In the pilot, Harness v0.1 and Plain 128K disagreed on 9 of 30 attempted pairs, a discordance rate $q \approx 0.30$. With $n \approx (z_{0.975} + z_{0.80})^2\, q/\delta^2$, detecting a 10-point difference δ at $\alpha = 0.05$ and 80% power needs about 240 pairs (about 320 with the interim looks below), and a 5-point difference about 940. A grid of 64 per cell detects only differences of roughly 25 points or more. The plan therefore extended the grid in rolling batches to 320 pairs for the primary comparison at small and medium. Seeds 2–3 were seen before the hypotheses were frozen and are exploratory; the *confirmatory* set is seeds 4–43 (8 tasks × 40 seeds = 320 pairs per size), with interim looks after seeds 17 (112 pairs) and 30 (216) and the final look at seed 43 (`evals/TEST-PLAN.md`, section 1b). The 64 large questions per harness arm (H4) have no plain comparator. 5-point effects are out of reach on this hardware.

Pairing, clustering and replicates. Every harness question is also answered by plain; plain-only seeds may estimate baseline accuracy but never count toward paired n. Variance is dominated by the task generator, so tasks are treated as a random effect: we report per-task effects, a task-weighted average and a cluster bootstrap by task, and note that generalising beyond these eight tasks needs more generators (the plan targets at least 16). About 10% of runs are same-question repeats, used to measure sampling noise and never added to n. Runs of other serving stacks or model variants are separate experiments, never pooled with `pluto`'s.

5.4 Metrics and statistics

Following the test plan in the repository (`evals/TEST-PLAN.md`), each row records how the run ended (answered, forced finish after a limit, does not fit, or not run because of a provider or transport failure, which is never scored). We report completion separately from accuracy. Accuracy is the exact-match rate (score 1.0 under the task's own scorer) with a Wilson 95% interval, and the mean score with a bootstrap 95% interval. Paired comparisons use McNemar's exact test on discordant cells and a paired bootstrap of the mean difference; a difference is reported as real only if the test says so. Cost is tokens in and out including sub-calls; latency is wall-clock time, reported as median and 95th percentile. Every number names its code commit, task-set version, grader version, seeds and profile. All tables are generated by `paper/analysis/make_tables.py` from the row files; none is typed by hand.

5.5 Hypotheses

The test plan requires each phase's hypothesis, arms, sample size, metric, test and stopping rule to be written down before the phase runs. The hypotheses below were frozen in `evals/TEST-PLAN.md` (section 1a, commit `692a62a`) on 2026-10-09, before any seed-4–9 score was examined. Those runs had started earlier that day, so the freeze precedes looking at their results, though not their collection.

H1 (primary; harness vs plain, within the window). At small and at medium size, exact-match accuracy of Harness v0.1 differs from Plain 128K on paired questions. Two-sided exact McNemar test with three looks at about 107, 213 and 320 pairs, using O'Brien–Fleming boundaries ($|z| > 3.471, 2.454, 2.004$; corrected 2026-10-10 before any look, see `evals/TEST-PLAN.md` section 1b). At 64 pairs (seeds 2–9) no look is reached, so that grid reports estimates with intervals and no test of H1.

H2 (secondary; configuration comparison). Harness v0.2 128K and Harness v0.1 differ in exact-match accuracy at small and at medium size (two-sided McNemar, unadjusted). The arms differ in prompt, limits and window together, so a difference cannot be attributed to delegation.

H3 (secondary; delegation). Harness v0.2 makes more sub-calls per question than v0.1 (one-sided paired Wilcoxon signed-rank on sub-calls per question).

H4 (beyond the window). At large size, where no plain arm fits, both harness arms report completion and exact-match accuracy with intervals; no comparison with plain is possible. This is an estimate, not a test.

H5 (secondary; sampling). For each arm, accuracy with the model's embedded sampling defaults differs from accuracy with the Qwen3 presets on the same questions (two-sided exact McNemar per arm). H5 also asks whether the directions of H1 and H2 hold under both settings. The two grids are never pooled.

H6 (secondary; map-reduce baseline). Fixed-chunk map-reduce with the same model as reader, deterministic prompts and one aggregation call differs from Harness v0.1 in exact-match accuracy. At large size it is the harness's only comparator (added 2026-10-10, section 1b, before any row of either arm).

H7 (secondary; REPL-only). Harness v0.1 with `llm_query` disabled and not advertised differs from Harness v0.1. A REPL-only score close to the harness's would mean the gain comes from code in a REPL, not from recursion (added with H6).

H1-small and H1-medium are two separate primary questions, each tested at its own $\alpha = .05$; Holm-adjusted p-values across the pair are reported as a sensitivity check, and no claim combines them. H2–H7 are secondary, labelled so, and H4 is an estimate rather than a test. The analysis plan (section 1b) fixes, before the first look: a task-clustered paired bootstrap and a GLMM with a task random intercept beside McNemar; leave-one-task-out estimates and the minimum detectable effect at each n; per-task, per-author and evidence-position breakdowns; an A/A noise floor from the two plain arms on identical input; completion, cost and latency as central results; and a failure taxonomy.

Ablations that attribute any H2 difference to the prompt, the turn cap or the REPL output limit are a separate, later phase (Section 10).

6 Results

This section reports H1, which is concluded at both sizes. The pre-registered procedure ran, stopped, was found to have been measuring a scorer artefact, and was recomputed under corrected scorers with the deviation disclosed; both records are given. H2–H7 are pre-registered and pending (Section 6.3). Every number below is generated from the run files by `paper/analysis/h1_analysis.py`; the markdown report it writes is `paper/results/h1-look3.md`, and `evals/TEST-PLAN.md` sections 1c–1f are the dated record.

6.1 Protocol deviations

Three deviations from the pre-registered plan occurred, all recorded in the test plan on the day they were found.

The seed-17 look was late. The first interim look was due when seed 17 finished (2026-10-15) and was not performed; the grid ran on to seed 30 before any look was computed (2026-10-16). Nobody read a score in between (only row counts and error rows), and the recorded result applies the pre-registered rule as written: H1-medium counted as stopped at the first look, H1-small at the second (section 1c).

A one-directional scorer defect. On 2026-10-19, while answering a multi-model review of the per-task table, we found that the frozen "round 3" scorers penalised answers that show their work: three or more dollar amounts anywhere in the text, a "not" or "or" near a second number, a name mentioned during the reasoning, a status history before the final status. The plain-128K arm shows its work; the harness answers `Name; $X`. All 54 plain MasterControl rows at seeds 4–30 named the right employee and the right total and scored 0.3; 22 of 41 plain Neuromancer zeros ended with the right amount written as `\boxed{\$12{,}632.19}`, which the regex could not read. The recorded H1 therefore could not be reported as a result (section 1d). Corrected "round 4" scorers were written to grade an answer's *final statement* after formatting normalisation, with a hedge defined as an explicit alternative inside that statement, and with the same code for both arms. Before any paper number was taken from them they had to: reproduce every stored answer key byte for byte; pass the 2,111-probe scorer battery in CI; have every row whose exactness they flipped relative to round 3 read by hand (111 rows, all upward, listed on the issue); and agree with the author's blind hand-grading of a 40-pair sample at least as well as round 3 did on every task. On that sample the author agreed with round 4 on 71 of 77 marks and with round 3 on 59 of 77, and with an independent blind LLM judge over all 432 pairs on 72 of 74. Recorded scores were never overwritten; the rescored files are separate and carry both scores per row. A further key defect in one task (Cerebex: the status was drawn before the wording that described it) corrected 13 keys with contexts byte-identical ("round 5", section 1d-iii); recorded rows were regraded, not re-run.

The stops were re-evaluated and both sizes ran to 320 pairs. Under the corrected scorers neither size crossed its boundary at the looks where the recorded procedure had stopped (Table 4), so, as the O'Brien–Fleming design allows for a look that does not cross, both sizes continued to the final look at seed 43. The decision for each size was recorded before its remaining rows were run (sections 1d-ii and 1e-i). The recomputed H1 is *not* the pre-registered test: its scorer was chosen after the data were seen. It is reported as the corrected estimate under the same pairing, looks and boundaries.

6.2 H1: harness v0.1 32K vs plain-128K

Small (confirmatory, final look). The harness and the single-call baseline differ, in the baseline's favour: 203 vs 229 exact of 320, $b = 22$, $c = 48$, $z = -3.11$, exact McNemar $p = .0025$ (Table 5). The GLMM agrees (arm odds ratio 0.46, $p = .002$). The task-clustered bootstrap does not: the exact-rate difference is -0.081 with a 95% interval of $[-0.294, 0.072]$, which includes zero, because the effect is one task. SHODAN (Table 6): harness 2 of 40, plain 32 of 40 ($b = 1$, $c = 31$). 39 of the 40 harness SHODAN rows hit the 20-turn cap and one the time limit; none submitted an answer.

Table 4: H1 at every look, as recorded under the original (round-3) scorers and as recomputed under the corrected scorers (round 4, with the round-5 Cerebex keys), on the same pairs. Boundaries $|z| > 3.471$, 2.454, 2.004 at 112, 216 and 320 pairs; positive z favours the harness. The recorded procedure stopped H1-medium at seed 17 and H1-small at seed 30; its seed-30 medium row is a supplementary estimate, not a test. Holm across the two primaries at the final look: small $p = .0025 \rightarrow .005$, medium $p = .040 \rightarrow .040$. Source: `evals/TEST-PLAN.md` sections 1c, 1e and 1f.

			Recorded (round 3)					Corrected (round 4 + 5)				
Look	Size	n	A	B	b/c	z	Result	A	B	b/c	z	Result
seed 17	small	112	65	49	32/16	+2.31	inside	70	76	11/17	−1.13	inside
seed 17	medium	112	51	32	24/5	+3.53	**stop**	63	55	11/3	+2.14	inside
seed 30	small	216	125	94	60/29	+3.29	**stop**	135	149	16/30	−2.06	inside
seed 30	medium	216	104	68	49/13	+4.57	suppl.	121	109	22/10	+2.12	inside
seed 43	small	320	not run					203	229	22/48	−3.11	**crosses, plain**
seed 43	medium	320	not run					177	162	31/16	+2.19	**crosses, harness**

Table 5: H1, harness v0.1 32K vs plain-128K, look 3 (boundary $|z| > 2.004$). Primary: every pair counts. Secondary and sensitivity sets per TEST-PLAN 1d. Bootstrap: task-clustered paired, 95%. GLMM: logit, arm fixed effect, task random intercept.

Size	Set	n	A ex.	B ex.	b/c	z	p
small	primary: every pair counts	320	203	229	22/48	−3.11	0.003
	secondary: plain does_not_fit pairs dropped	320	203	229	22/48	−3.11	0.003
	sensitivity: plain length rows dropped	320	203	229	22/48	−3.11	0.003
	sensitivity: harness turn-cap/timeout rows dropped	228	178	174	15/11	+0.78	0.557
	sensitivity: both arms finished	228	178	174	15/11	+0.78	0.557
	bootstrap Δexact -0.081 [-0.294, 0.072]; GLMM arm OR 0.46, $z = -3.09$, $p = 0.002$, task SD 1.98						
medium	primary: every pair counts	320	177	162	31/16	+2.19	0.040
	secondary: plain does_not_fit pairs dropped	280	173	162	27/16	+1.68	0.126
	sensitivity: plain length rows dropped	276	174	162	28/16	+1.81	0.096
	sensitivity: harness turn-cap/timeout rows dropped	202	156	138	28/10	+2.92	0.005
	sensitivity: both arms finished	197	152	138	24/10	+2.40	0.024
	bootstrap Δexact +0.047 [-0.047, 0.163]; GLMM arm OR 1.72, $z = +1.99$, $p = 0.046$, task SD 3.17						

Leaving SHODAN out gives $z = +0.65$ ($p = .63$); leaving out any other task keeps z between -2.65 and -4.63. This is a harness failure on that task under the v0.1 turn and output budget, not a scoring artefact: the plain answers are exact key matches. Dropping the harness's turn-cap and time-limit rows (a sensitivity set, post-treatment) gives $n = 228$, $z = +0.78$. The discordance rate $b + c = 70$ of 320 (0.22) sits near the noise floors: the A/A floor (plain-32K vs plain-128K on identical small input, where both fit) is 10 of 56 cells (0.18), and the temperature-0 replay floor on the same stack is 5 of 48 rows (0.10).

Medium (confirmatory, final look). The arms differ, in the harness's favour: 177 vs 162 exact, $b = 31$, $c = 16$, $z = +2.19$, $p = .040$. The margin is one pair: with $b = 30$, $c = 17$ the look would have been inside the boundary. The bootstrap interval ($+0.047$, $[-0.047, 0.163]$) includes zero; the GLMM is at the edge (odds ratio 1.72, $p = .046$). The effect is mostly one task: Neuromancer,

Table 6: H1 per task, look 3, primary set (every pair counts; 40 pairs per task and size). A = harness v0.1 32K, B = plain-128K; b = A exact and B not, c = the reverse; $z = (b - c)/\sqrt{b + c}$, exact McNemar p.

	small						medium					
Task	A ex.	B ex.	b	c	z	p	A ex.	B ex.	b	c	z	p
Cerebex	37	38	2	3	−0.45	1.000	29	35	1	7	−2.12	0.070
GLaDOS	19	11	12	4	+2.00	0.077	4	0	4	0	+2.00	0.125
MasterControl	39	40	0	1	−1.00	1.000	40	40	0	0	+0.00	1.000
Multivac	32	37	1	6	−1.89	0.125	32	32	5	5	+0.00	1.000
Neuromancer	35	34	3	2	+0.45	1.000	32	17	18	3	+3.27	0.001
SELMA	39	36	3	0	+1.73	0.250	39	37	3	1	+1.00	0.625
SHODAN	2	32	1	31	−5.30	0.000	0	0	0	0	+0.00	1.000
TheDixieFlatline	0	1	0	1	−1.00	1.000	1	1	0	0	+0.00	1.000

Table 7: H1 completion, cost and latency per arm, look 3 (320 pairs per size). Finished = a submitted answer; tokens are means per question, sub-calls included; latency is wall-clock seconds. Below each size, how the unfinished rows ended.

Size	Arm	n	Finished	s p50	s p95	Prompt tok	Compl. tok	Sub-calls	Turns
small	harness	320	228 (0.71)	398	1429	70116	5620	0.71	13.5
	plain	320	320 (1.00)	204	562	18191	2866	0.00	1.0
	forced finish: time limit: harness 5, plain 0								
	forced finish: turn cap: harness 87, plain 0								
medium	harness	320	202 (0.63)	421	1532	79788	6201	0.55	14.6
	plain	320	236 (0.74)	580	1385	79905	5617	0.00	1.0
	forced finish: time limit: harness 7, plain 0								
	forced finish: turn cap: harness 111, plain 0								
	not_run: does_not_fit: harness 0, plain 40								
	output-length cap: harness 0, plain 44								

harness 32 vs plain 17 ($b = 18$, $c = 3$), where 15 of the 18 plain losses are normal finishes with a wrong answer. Leaving Neuromancer out gives $z = 0.00$. The crossing also depends on the 40 medium GLaDOS contexts that do not fit plain-128K: the pre-registered primary counts them against plain, which is the question H1 asks (can the harness answer what the baseline cannot read), and the harness gets 4 of them; the pre-registered secondary set that drops them gives $n = 280$, $z = +1.68$, $p = .126$, inside the boundary. Cerebex goes the other way (harness 29, plain 35). SHODAN is 0 of 40 for both arms: plain hits the output-length cap on all 40, the harness the turn cap on 37 and the time limit on 3. Without Multivac, whose corrected scorer still under-credits some right answers (section 1d-ii), $z = +2.47$, $p = .020$.

What H1 says. The two sizes point in opposite directions, and each result rests on one or two tasks. Small favours the baseline because of SHODAN; medium favours the harness because of Neuromancer and the GLaDOS inputs that do not fit the baseline's window. Both primaries cross their final boundaries and both survive Holm, but neither supports a general claim that the harness is better or worse than long-context prompting on this model: with eight tasks as the clusters,

both bootstrap intervals include zero, and both discordance rates are within a factor of two of the measurement noise floor.

Completion, cost and latency. (Table 7.) At small size the harness submitted an answer on 228 of 320 questions (87 turn caps, 5 time limits) against the baseline's 320, took about twice the wall-clock time at the median (398 vs 204 s) and about 3.9 times the prompt tokens (70,116 vs 18,191 per question, sub-calls included), with 0.71 sub-calls and 13.5 turns per question. At medium size the harness finished 202 of 320 (111 turn caps, 7 time limits) and the baseline 236 (40 contexts did not fit, 44 hit the 16,384-token output cap: all 40 SHODAN rows and 4 Neuromancer rows); median latency was 421 s against 580 s, and prompt tokens were level (79,788 vs 79,905) because the baseline's prompt is the whole document. Latency figures are for shared hardware with two serving slots (Section 9).

Construct validity: what a non-LLM shortcut scores. Per the plan's construct-validity check (section 1b), two deterministic solvers with no model and no answer key were run on every confirmatory question: the last-mentioned candidate of the answer's type, and a keener regex on the question's key terms, each graded by the task's own scorer. On seeds 4–43 the regex solves MasterControl 40 of 40 at both sizes, Multivac 40 of 40 (the last-mentioned status alone does), and SELMA 36 of 40 small and 39 of 40 medium; those three tasks are therefore flagged as measuring search rather than multi-hop reasoning, and the per-task table should be read with that in mind (they are also the three tasks both arms answer almost perfectly). The other five tasks score at most 7 of 40 (Cerebex small) and mostly 0; GLaDOS, Neuromancer, SHODAN and TheDixieFlatline are not solvable by either shortcut (`evals/independent/shortcut_solver.py`).

6.3 H2–H7: pending

H2, H3. [**PENDING:** Configuration comparison and delegation (harness v0.2 128K vs v0.1), seeds 4–9 and onward, reported as estimates with task-clustered intervals and labelled secondary. The two harness v0.2 rows lost to the server-side "malformed tool call" rejection (Section 9) score 0 with a sensitivity set excluding them.]

H4. [**PENDING:** Large-size completion and exact-match accuracy per harness arm with intervals (64 questions per arm). An estimate, not a test, and not a capability claim.]

H5. [**PENDING:** The native-sampling grid, pre-registered 2026-10-25 (section 1g), runs after H1 and is never pooled with the preset grid.]

H6, H7. [**PENDING:** The map-reduce and REPL-only baseline arms, pre-registered 2026-10-25 (section 1h), run after H5 from a separate checkout at a recorded commit.]

Second model. A confirmatory H1 on a second local model (Laguna S 2.1) and a frontier replication kit with the paper's own settings were pre-registered on 2026-10-29 (sections 1i and 1j); neither has rows yet, and neither is part of the claims above.

7 Pilot and lessons learned (preliminary)

Everything in this section is preliminary. It comes from development runs and a 16-question-per-cell pilot, and it is reported because the lessons shaped the design above, not because it supports any claim about the method.

7.1 Lesson 1: the baseline was selected on the answer key

Our first evaluations made two plain calls per question (thinking on and thinking off), scored both against the key, and kept the better one, while the harness got one attempt. That is an oracle. On the 16 small questions of a run on the round-2 task set, plain was exact on 12 as best-of-2, but on 9 with thinking on alone and on 8 with thinking off alone. The harness was exact on 6. The headline "plain beats the harness below the window" from those runs was therefore partly an artefact of selection. Since then plain is one call with no selection, and the two variants are separate arms when needed.

7.2 Lesson 2: graders moved results, and not evenly

The first two evaluation runs used tasks that later audits found defective (Section 4). Between the first run and a re-run after the round-2 fixes, harness exact answers on the four repaired tasks went from 6 of 20 to 12 of 20, and on the four unchanged tasks from 5 of 20 to 6 of 20: the apparent improvement came from repairing tasks, not from changing the harness. In the pilot below, the round-3.1 scorer fix changed four Plain 128K cells from 0.2 or 0.3 to 1.0, one cell in each v0.2 harness arm and in Plain 32K, and none in Harness v0.1. A grader defect thus cost the baseline more than the harness. Four Plain 128K answers could not be re-scored because the row files stored only their first 2,000 characters; they keep their original scores.

7.3 Lesson 3: a prompt fix for over-delegation suppressed delegation

The paper's Qwen-specific prompts add a warning against sub-calls because Qwen3-Coder over-delegated [25, App. C]. We copied that warning into v0.1. Our model has the opposite tendency: on the round-2 re-run only 7 of 40 harness rows made any sub-call, and 15 of 40 used all 20 turns, mostly reading slices of the document itself and writing ad hoc parsers. Version 0.2 removes the warning and lifts our limits. In the pilot it delegated far more (sub-calls on 14 of 16 small questions at 128K, against 3 of 16 for v0.1; Table 10) without a detectable change in accuracy (Table 9).

7.4 The pilot grid

The pilot ran the five arms of Table 3 on seeds 2–3 of the round-3 task set, graded with the round-3.1 scorers (Tables 8, 10 and 9). Harness v0.1 was exact on 11/16 small and 9/16 medium questions;

Table 8: Accuracy per arm and size. **Preliminary pilot**: seeds 2, 3, n=16 cells per arm and size, round-3.1 grader. Not a test of any hypothesis. Exact = score 1.0 under the task's own `score()`; a plain prompt that does not fit the window (DNF) scores 0. Intervals: Wilson 95% for exact, bootstrap 95% for the mean.

Size	Arm	Exact	95% CI	Mean	95% CI	DNF
Small	Plain 32K	9/16	[0.33, 0.77]	0.60	[0.37, 0.83]	2
	Plain 128K	8/16	[0.28, 0.72]	0.56	[0.34, 0.78]	0
	Harness v0.1	11/16	[0.44, 0.86]	0.71	[0.49, 0.90]	0
	Harness v0.2 32K	10/16	[0.39, 0.82]	0.68	[0.45, 0.88]	0
	Harness v0.2 128K	10/16	[0.39, 0.82]	0.69	[0.48, 0.89]	0
Medium	Plain 32K	0/16	[0.00, 0.19]	0.00	[0.00, 0.00]	16
	Plain 128K	7/16	[0.23, 0.67]	0.47	[0.24, 0.71]	2
	Harness v0.1	9/16	[0.33, 0.77]	0.62	[0.39, 0.83]	0
	Harness v0.2 32K	7/16	[0.23, 0.67]	0.49	[0.27, 0.72]	0
	Harness v0.2 128K	8/16	[0.28, 0.72]	0.57	[0.34, 0.78]	0

Plain 128K on 8/16 and 7/16. Pooled over both sizes the discordant cells split 7/2 (McNemar $p = 0.18$): *no detectable difference at this sample size.* Harness v0.2 128K against Plain 128K split 5/2 ($p = 0.45$), and against Harness v0.1 5/3 ($p = 0.73$). The only significant pilot contrasts are those where the plain prompt does not fit at all (Plain 32K at medium size), which are true by construction. A few bootstrap intervals on paired mean differences exclude zero (Table 9); at $n = 16$, uncorrected for the number of comparisons, we treat none of them as evidence. Sixteen questions per cell cannot distinguish differences of a few questions; the pilot's discordance rate is what sized the main study (Section 5).

7.5 Side runs with other planners

Three side runs, on the round-3 task set with v0.1 settings, are summarised in Table 11. Their plain rows predate Lesson 1 and are best-of-2.

The paper's RLM-Qwen3-8B as planner. With the compatibility mode, the released checkpoint (a community GGUF conversion on a local llama.cpp server) planned and Flash-Next on `pluto` read. It was exact on 1 of 16 small, 4 of 16 medium and 0 of 8 large questions. It committed early (median 2 turns), and 14 of 40 rows ended at the time limit. *These conditions differ materially from the paper's*: the planner window was 16,384 tokens rather than 32K; the reader was a different model on slower, shared hardware rather than Qwen3-8B; our sub-call caps and error messages are ones the checkpoint never saw in training; the small rows and the medium and large rows ran at different harness commits; and the reader's server configuration changed during the large rows. Nor does it test the paper's actual 8B claim, that fine-tuning helps: there was no base Qwen3-8B arm. The result says nothing about RLM-Qwen3-8B under its native conditions, which remain to be tested (Section 10).

Table 9: Paired comparisons on identical cells. **Preliminary pilot**: seeds 2, 3, n=16 cells per arm and size, round-3.1 grader. Not a test of any hypothesis. b = cells only A got exact, c = cells only B got exact; p = McNemar's exact two-sided test on (b, c). Δ = paired mean-score difference A−B with a bootstrap 95% interval. Uncorrected for multiple comparisons.

Size	A	B	A ex.	B ex.	b/c	p	Δ mean [95% CI]
Small	Harness v0.1	Plain 128K	11/16	8/16	5/2	0.45	+0.15 [-0.14, 0.42]
	Harness v0.1	Plain 32K	11/16	9/16	5/3	0.73	+0.11 [-0.23, 0.42]
	Harness v0.2 128K	Plain 128K	10/16	8/16	2/0	0.50	+0.13 [0.01, 0.29]
	Harness v0.1	Harness v0.2 128K	11/16	10/16	3/2	1.00	+0.02 [-0.23, 0.25]
	Harness v0.1	Harness v0.2 32K	11/16	10/16	1/0	1.00	+0.04 [0.00, 0.11]
Medium	Harness v0.1	Plain 128K	9/16	7/16	2/0	0.50	+0.14 [0.03, 0.30]
	Harness v0.1	Plain 32K	9/16	0/16	9/0	0.004	+0.62 [0.40, 0.83]
	Harness v0.2 128K	Plain 128K	8/16	7/16	3/2	1.00	+0.09 [-0.14, 0.33]
	Harness v0.1	Harness v0.2 128K	9/16	8/16	2/1	1.00	+0.05 [-0.11, 0.24]
	Harness v0.1	Harness v0.2 32K	9/16	7/16	3/1	0.62	+0.12 [-0.07, 0.34]
Pooled	Harness v0.1	Plain 128K	20/32	15/32	7/2	0.18	+0.15 [-0.01, 0.31]
	Harness v0.1	Plain 32K	20/32	9/32	14/3	0.01	+0.37 [0.14, 0.57]
	Harness v0.2 128K	Plain 128K	18/32	15/32	5/2	0.45	+0.11 [-0.02, 0.25]
	Harness v0.1	Harness v0.2 128K	20/32	18/32	5/3	0.73	+0.03 [-0.11, 0.18]
	Harness v0.1	Harness v0.2 32K	20/32	17/32	4/1	0.38	+0.08 [-0.02, 0.20]

Poolside Laguna S 2.1. With this hosted model the harness was exact on 5 of 16 small, 2 of 16 medium and 3 of 8 large questions, and 28 of 40 runs hit the 20-turn cap. Laguna writes its native tool-call syntax (`<tool_call>repl ...</tool_call>`) as message text, which the fence parser originally ignored, turning those turns into no-ops; it also sometimes followed a call with REPL output it had invented. The parser now accepts the form and discards the rest of a reply after the first such call. Re-running 24 cells after the fix moved exact answers from 5 to 6: the fix restored the protocol but did not detectably change accuracy. A provider quota exhaustion during this run produced rows recorded as scores of zero; they were quarantined, and the runner now must record provider failures as not run.

Llama 3.3 70B, and Python versus Rust. A separately developed Rust port of the harness scored poorly with Llama 3.3 70B. To separate the language of the harness from the model, we ran the Python harness with the same model, provider and sampling on the same GLaDOS cells: neither the harness nor plain was exact on any of the four cells (mean scores 0.10 for both modes). [**TODO:** Insert the Rust port's per-cell scores from its repository and confirm they match; the claim of equal scores comes from the session log and is not yet verified against the Rust run files.] The working conclusion is that the model, not the harness's language, was the bottleneck.

7.6 Practicality

Wall-clock time, not accuracy, was the binding constraint throughout. Sub-calls run one or two at a time on a serial server; a plain 128K call on a medium document spends several minutes reading its prompt (median [**PENDING:** main-grid median] s); and the 95th-percentile harness run under

Table 10: Completion, delegation, cost and latency over attempted rows. **Preliminary pilot**: seeds 2, 3, n=16 cells per arm and size, round-3.1 grader. Not a test of any hypothesis. Ans. = the model finished on its own (`FINAL`, `FINAL_VAR`, the answer dict, or a plain reply that stopped normally); Lim. = turn cap, time limit, error limit, context overflow or output-length cap (the harness then takes a forced finish). Deleg. = rows with at least one sub-call; Sub-calls and Turns are means per attempted row. Tokens include sub-calls. Seconds are wall clock on shared, serial hardware.

Size	Arm	Ans.	Lim.	Deleg.	Sub-calls	Turns	Med. tokens	Med./p95 s
Small	Plain 32K	14/14	0/14	–	–	–	17,870	113/226
	Plain 128K	16/16	0/16	–	–	–	18,873	134/263
	Harness v0.1	12/16	4/16	3/16	0.75	13.50	55,137	128/515
	Harness v0.2 32K	14/16	2/16	10/16	1.62	12.56	43,277	206/1,201
	Harness v0.2 128K	16/16	0/16	14/16	2.44	12.25	67,688	252/1,856
Medium	Plain 32K	0/0	0/0	–	–	–	–	–/–
	Plain 128K	12/14	2/14	–	–	–	88,699	449/607
	Harness v0.1	10/16	6/16	4/16	0.38	14.75	103,628	236/459
	Harness v0.2 32K	14/16	2/16	7/16	1.44	16.50	100,321	371/2,739
	Harness v0.2 128K	15/16	1/16	10/16	2.06	16.75	132,842	374/3,245

v0.2 took close to an hour in the pilot (Table 10). Hardware failures cost more time than any single experiment: on 2026-10-09 the RTX 3060 fell off the PCIe bus (NVIDIA Xid 79) for about 2.5 hours, turning 14 harness rows into errors that the runner then recorded as zero scores; a network interface failed later the same morning. All affected rows were removed (the originals are kept) and re-run. This matches the paper's own account. Its limitations section says that "future strategies involving asynchronous sub-calls and sandboxed REPLs can potentially significantly reduce the runtime and inference cost of RLMs" [25, §7], and its negative results say: "RLMs without asynchronous LM calls are slow. We implemented all sub-LM queries naively as blocking / sequential calls, which caused our RLM experiments to be slow, especially compared to just the base model" [25, App. B].

7.7 Process errors

For completeness, errors in how the study was run, each fixed and recorded in the repository: a pull request merged without waiting for CI broke the main branch on one Python version; the first validator checked only that a key scores itself, and so missed the answerability defects; merging a pull request moved the experiment's checkout mid-run (a scorer-only change), crashing one arm, which was resumed; output tokens were reserved twice in the 8B runs, producing context-overflow rows that were discarded and re-run; and turn-by-turn trajectories of three early development runs were lost when temporary worktrees were cleaned (their summary files survive).

8 Discussion

On H1 the answer to the paper's question is: not in general, on this model and these tasks. Within the window (small size) the single-call baseline was more accurate, and the whole difference is one task on which the v0.1 harness could not finish inside its 20 turns. Beyond the baseline's comfortable

Table 11: **Preliminary** pilot side runs on the round-3 task set (seeds 0–1 at small and medium, seed 0 at large; the Llama run is GLaDOS only), with harness v0.1 settings. Their plain rows still used the best-of-2 rule of Section 7, so plain is an upper bound. Exact / cells. Observations, not tests.

Setup	Size	Harness exact	Plain exact (best-of-2)
RLM-Qwen3-8B planner	small	1/16	–
	medium	4/16	–
	large	0/8	–
Laguna S 2.1	small	5/16	4/16
	medium	2/16	2/16 (2 DNF)
	large	3/8	–
Llama 3.3 70B (GLaDOS only)	small	0/2	0/2
	medium	0/2	0/2 (2 DNF)
Laguna, parser-fix reruns	24 cells	5/24 before, 6/24 after	–

reach (medium size) the harness was more accurate, and the difference is mostly one task plus the inputs the baseline could not read at all. Both effects are statistically real under the pre-registered procedure and both are fragile under the clustered analysis the plan also pre-registered. The honest summary is task-conditional: where a question needs the model to carry a state through many records (SHODAN's per-office credits, 18 shipments), v0.1's turn and output budget is the binding constraint, not recursion; where the document does not fit, the harness is the only arm that answers at all; and on the five tasks both arms solve nearly perfectly, including the three a regex also solves, the harness pays twice the latency and several times the tokens for the same answer. Whether a larger budget (H2) or more delegation (H3) changes the small-size picture, and whether code in a REPL without recursion does as well (H7), are the pending secondary questions.

The scorer episode is part of the result. A frozen, battery-tested grader still carried a one-directional bias that the pre-registered procedure could not see, because the two arms write their answers differently. It was found only when the per-task table was put beside raw answers in a review. The corrected scorers reversed the small-size result and moved the medium stop by a look. We report both so that the reader can see the size of a measurement choice next to the size of the effect.

Two points can be made now, because they concern design rather than results. First, our tasks are mostly multi-hop chains whose deciding evidence is a handful of records, which in the taxonomy of Goldman et al. [9] is low-dispersion work. The RLM paper's largest gains are on information-dense aggregation such as OOLONG and OOLONG-Pairs, and its earlier versions observe that "the base LM outperforms RLM in the small input context regime" [25, v2, §4]. A null result on our tasks would therefore not contradict the paper. Second, the pilot shows that measurement choices (baseline selection, grader defects, a copied prompt patch) can move a comparison of this kind by several questions in 16, which is the size of the effects under study.

9 Threats to validity

- **Synthetic data.** All tasks are generated corporate records written by other models. They may share conventions that favour or penalise code-based reading, and they are not real documents.
- **One main model, of a different architecture from the paper's.** Results are for one quantized, abliterated open-weight model on one server. Its architecture (`qwen4exp`) differs from every Qwen model in the RLM paper, so within-model comparisons (harness vs plain, v0.1 vs v0.2) are valid but comparisons with the paper are cross-architecture. Abliteration and 4-bit quantization may change behaviour beyond refusals.
- **Sampling and templates.** All runs used Qwen3 model-card sampling presets rather than the model's embedded defaults, and the thinking switch was assumed to behave on this architecture's chat template as on Qwen3's. Neither is verified yet.
- **Small samples, few tasks.** The pilot has 16 questions per cell; the main study targets 320 pairs for the primary comparison but starts at 64. With eight task generators, effective sample size is limited by between-task variance. Run-to-run noise is not yet measured.
- **Shared hardware.** `pluto` served other work during the study; latency figures reflect contention, two parallel slots, and a server configuration that changed on 2026-10-08.
- **Code changes during runs.** Two arms crossed a scorer-only commit; side runs crossed harness commits. Every row records its commit, and pilot rows were re-graded with a single grader version.
- **Outages and provider failures.** Rows lost to hardware or provider failures were removed and re-run, with originals kept; before that rule existed, such rows were scored as zero.
- **Grader residuals.** The scorers pass a 2,111-probe battery but are still programmatic. Human spot-checks of 10% of graded answers are planned, not done. Four plain answers in the pilot could not be re-graded.
- **Author and assistant overlap.** The harness and its prompts were developed with Claude models, and Claude also orchestrated the evaluation. The task generators were deliberately sourced elsewhere, but the analysis pipeline was not independently reimplemented.
- **Generator-author familiarity.** The eight generators were written by eight other models, and a Qwen-family lane was excluded because the system under test is Qwen-branded. Lineage is still a threat: training-data overlap between a generator's author and the model under test could make that task's conventions more familiar to it, in either arm, and we cannot measure that. The per-author breakdown is reported with the per-task one so the reader can see which models' tasks drove each result.
- **The scorer defect.** The frozen scorers penalised one arm's answer style (Section 6.1). The corrected scorers were chosen after the data were seen, so the recomputed H1 is a corrected

estimate, not the pre-registered test; the validation steps, the hand-read flips and the agreement with blind human grading are the evidence that the correction is not itself a bias. Multivac's corrected scorer still under-credits some right answers in both arms, so medium is reported with and without it.

- **A server-side rejection counted as a model failure.** In fence mode the serving engine returned HTTP 400 "malformed tool call" when the model emitted tool-call-shaped text instead of a code fence; two harness v0.2 rows (Multivac, small, seeds 4 and 5) died this way. By a decision recorded before any score was read they count as wrong, labelled `malformed_tool_call`, and every analysis containing them is also reported with them excluded. No H1 row is affected.
- **Three tasks are search.** A non-LLM regex solves MasterControl, Multivac and SELMA almost perfectly (Section 6.2). Results on those tasks measure whether an arm can find and copy a value, not multi-hop reasoning; they are kept in the primary because the plan put them there, and the per-task table lets a reader set them aside.

10 Future work

The test plan orders the remaining work as follows. (1) H2–H7 as pre-registered (Section 6.3), and a diagnostic reading of the SHODAN trajectories to classify the turn-cap failures; any harness fix is a new arm with its own rows. (2) Validity checks: question-only and evidence-deleted controls, counterfactual twins, independent hand reconstruction of keys, a 10% human grading audit, and recovery of answers reached but not submitted. (3) Instrumentation of completion and of what each arm actually received. (4) A one-factor-at-a-time decomposition of v0.1 versus v0.2 (prompt, turn cap, REPL output limit), plus a check at matched total compute. (5) Oracle interventions that give readers the correct evidence or replace their outputs, and a BM25 retrieval baseline. (6) Coverage: unanswerable questions mixed 50/50, a size sweep from 16K to 1M tokens to locate any crossover, paraphrased questions, evidence position and chunk-boundary placement, generator-convention sensitivity, aggregation and long-output tasks, contradictions, real documents, and a reproduction of a paper benchmark (OOLONG or BrowseComp-Plus). (7) A replication of the paper's fine-tuning claim: base Qwen3-8B against RLM-Qwen3-8B as planner in the same harness and on the same questions, under native conditions (32K window, 20,000-character REPL outputs, Qwen3-8B reader, with a Flash-Next-reader variant as a separate arm), checked first on the paper's own tasks against its published numbers. This is also the same-architecture anchor that the main model cannot provide. (8) On rented GPUs: planner/reader pairings of different sizes. Also planned: the repeat of the main grid with the model's embedded sampling defaults (H5), at least eight more task generators from outside models, and a frontier model as a ceiling.

Provenance of the system under test

Table 12 summarises `paper/provenance.md`, which records every hash and command needed to rebuild the stack the numbers come from.

Table 12: What was tested. Full values, including the four GGUF shard hashes, the tokenizer and chat-template hashes, the three-line local engine patch and the build flags, are in `paper/provenance.md`.

Item	Value
Model	Qwen3.8 Flash Next, GGUF `general.architecture = qwen4exp`; 48 blocks, 512 experts, 10 active, 262,144-token native context
Weights	Huihui.ai abliterated build, Unsloth Dynamic `UD-Q4_K_XL`, 4 shards, 103.7 GiB; Hugging Face `huihui-ai/Huihui-Qwen3.8-Flash-Next-abliterated-GGUF`, shards unchanged from revision `7e3bfc316b88` (2026-09-06) through `4fdf4dc4d851`, deleted from the repository 2026-10-11 (pin a revision at or before `4fdf4dc4d851`)
Draft head	multi-token-prediction head from the same release, used for speculative decoding (`-spec 4 -spec-min-p 0.5`)
Tokenizer	GGUF BPE (`tokenizer.ggml.pre = qwen35`); chat template from the GGUF, 9,993 chars, applied server-side; the harness never tokenises (`ceil(chars/3.5)` estimates)
Engine	Strata, V100 fork `jmnargi/Strata-V100` commit `a4b679b1` (2026-10-05) with one local patch (skip the architecture check on continuation shards); ggml from llama.cpp `3cf0325`; CUDA 12.9, driver 580.159.04
Server	`strata -serve -max-context 262144 -kv-resident 65536 -kv int8 -batch 2`, int8 KV cache, 65,536 tokens resident, two slots, since 2026-10-08 11:55 CDT; every row in `runs/exp-v02` ran under it
Sampling	per request, from the harness profile: root temperature 0.6, top-p 0.95, top-k 20, thinking on; sub-calls 0.7, 0.8, 20, presence penalty 1.0, thinking off
Hardware	`pluto`: Ryzen 5 5600X, 125 GiB RAM, RHEL 9.8; Tesla V100 32 GB (PCIe x16, dense layers and most of the expert cache), RTX 3060 12 GB (external dock, x4), Tesla P100 16 GB (x4); non-resident experts and the n-gram table (about 26.8 GiB) in system RAM, about 89% expert-cache hit rate during decode
Scorers	`evals/independent/manifest.json`: `active` = round 4 (final-statement scoring) with round-5 Cerebex keys, sha256-pinned; every run file records the set that scored it

Code and data availability

The harness, the task generators with their full provenance (brief, verbatim answers, every repair round and the audit), the runner and this paper's analysis script are at https://github.com/CryptoJones/ReCLamO-Harness. Run row files and trajectories are archived off-machine and are not yet in the repository. [**TODO:** Publish the exp-v02 row files and trajectories (scrubbed of host addresses) with the final version.]

AI assistance disclosure

AI systems are not authors of this paper. Claude (Anthropic; Opus models) orchestrated the project, analysed results, wrote code, and drafted text including this manuscript and its analysis script. Claude Fable (Anthropic) wrote the v0.1 implementation of the harness. MiniMax-M3 repaired task generators and, together with a roundtable of non-Anthropic models (GLM, Mistral, DeepSeek, Nemotron, GPT, Gemini, Grok, Hermes and Qwen families), authored the independent test generators and reviewed the evaluation design (Grok's lane failed to authenticate for the design review), through the author's multi-model review harness [7], also used in Clark [6]. A Qwen-family

lane reviewed the design but did not author a test generator, because the system under test is also a Qwen-branded model. A separately developed Rust port of the harness, used only for the cross-check in Section 7, was written by MiniMax-M3. Model outputs were checked against the repository, the run files and the cited sources before use. The author directed the work, made the decisions recorded in the repository, and takes responsibility for the content, including any errors.

Acknowledgments

The author thanks Alex L. Zhang for encouragement: on seeing the harness he replied, "This is awesome! You should do a write up of your findings." This draft is that write-up, in progress. Any errors in reproducing the RLM method are the author's, not the original authors'.

Declarations

Funding: none. **Competing interests:** none.

References

[1] Y. Bai, S. Tu, J. Zhang, H. Peng, X. Wang, X. Lv, S. Cao, J. Xu, L. Hou, Y. Dong, J. Tang, and J. Li. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2025.

[2] A. Bertsch, A. Pratapa, T. Mitamura, G. Neubig, and M. R. Gormley. Oolong: Evaluating long context reasoning and aggregation capabilities, 2025.

[3] H. Chen, R. Pasunuru, J. Weston, and A. Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading, 2023.

[4] Z. Chen, X. Ma, S. Zhuang, P. Nie, K. Zou, A. Liu, J. Green, K. Patel, R. Meng, M. Su, S. Sharifymoghaddam, Y. Li, H. Hong, X. Shi, X. Liu, N. Thakur, C. Zhang, L. Gao, W. Chen, and J. Lin. BrowseComp-Plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025.

[5] Z. Chen, X. Wu, J. Jia, C. Gao, Q. Fu, D. Zhang, and S. Hu. LongBench Pro: A more realistic and comprehensive bilingual long-context evaluation benchmark, 2026.

[6] Aaron Kingsley Clark. The veto variable: Human override as a goal-independent cost term, 2026.

[7] Aaron Kingsley Clark. FlatlineRoundtable: an open-source harness for structured multi-model review. https://github.com/CryptoJones/FlatlineRoundtable, 2026.

[8] Aaron Kingsley Clark. ReCLamO-Harness: an open-source recursive language model harness for local models. https://github.com/CryptoJones/ReCLamO-Harness, 2026. Apache-2.0.

[9] O. Goldman, A. Jacovi, A. Slobodkin, A. Maimon, I. Dagan, and R. Tsarfaty. Is it really long context if all you need is retrieval? towards genuinely difficult long context NLP, 2025.

[10] K. Hong, A. Troynikov, and J. Huber. Context rot: How context degradation affects LLM performance, 2025. Technical report. TODO: confirm publisher and URL.

[11] C. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models?, 2024.

[12] MIT OASYS. rlm-qwen3-8b-v0.1 (model card). https://huggingface.co/mit-oasys/rlm-qwen3-8b-v0.1, 2026. Uploaded 2026-01-15; MIT license.

[13] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems, 2024.

[14] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[15] P. Schroeder, N. Morgan, H. Luo, and J. Glass. THREAD: Thinking deeper with recursive spawning, 2025.

[16] W. Sun, M. Lu, Z. Ling, K. Liu, X. Yao, Y. Yang, and J. Chen. Scaling long-horizon LLM agent via context-folding, 2025.

[17] K. Vodrahalli, S. Ontanon, N. Tripuraneni, et al. Michelangelo: Long context evaluations beyond haystacks via latent structure queries, 2024.

[18] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji. Executable code actions elicit better LLM agents, 2024.

[19] X. Wu, K. Li, Y. Zhao, L. Zhang, L. Ou, H. Yin, Z. Zhang, X. Yu, D. Zhang, Y. Jiang, P. Xie, F. Huang, M. Cheng, S. Wang, H. Cheng, and J. Zhou. ReSum: Unlocking long-horizon search intelligence via context summarization, 2025.

[20] A. Yang et al. Qwen3 technical report, 2025.

[21] H. Yu, T. Chen, J. Feng, J. Chen, W. Dai, Q. Yu, Y. Zhang, W. Ma, J. Liu, M. Wang, and H. Zhou. MemAgent: Reshaping long-context LLM with multi-conv RL-based memory agent, 2025.

[22] Alex L. Zhang. rlm-minimal. https://github.com/alexzhang13/rlm-minimal, 2025. MIT license. TODO: confirm year.

[23] Alex L. Zhang. rlm: Recursive language models. https://github.com/alexzhang13/rlm, 2026. MIT license. Tag v1.0.0, commit 18a9368 (2026-01-12), and main as of 2026-10.

[24] Alex L. Zhang and Omar Khattab. Language model harnesses are compositional generalizers. https://alexzhang13.github.io/blog/2026/harness/, July 2026.

[25] Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2025. v1 December 2025; v2 2026-01-28; v3 2026-05-11. Accepted to NeurIPS 2026 per the authors' blog.

[26] A. Zhu, L. Dugan, and C. Callison-Burch. ReDel: A toolkit for LLM-powered recursive multi-agent systems, 2024.